Name:R.K Ayushman Singh

Roll no :261100690014

LINUX ASSIGNMENT D2 AWK

Input file used: auth.log (columns: Time, SourceIP, Event, Severity, Attempts)

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ cat auth.log
Time SourceIP Event Severity Attempts
08:05 172.16.4.21 Login_Success LOW 1
08:12 172.16.4.37 Login_Failed MEDIUM 4
08:18 10.10.2.5 Port_Scan HIGH 12
08:26 192.168.0.44 Login_Failed HIGH 6
08:31 10.10.2.9 Malware_Alert CRITICAL 3
08:40 172.16.4.21 File_Access LOW 2
08:47 192.168.0.51 Login_Failed MEDIUM 9
08:55 10.10.2.5 Port_Scan HIGH 5

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

Write an AWK program for each question. Ignore the header row where appropriate.

1. Print the source IP and event from every record.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {print $2, $3}' auth.log
172.16.4.21 Login_Success
172.16.4.37 Login_Failed
10.10.2.5 Port_Scan
192.168.0.44 Login_Failed
10.10.2.9 Malware_Alert
172.16.4.21 File_Access
192.168.0.51 Login_Failed
10.10.2.5 Port_Scan

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

2. Display the time, source IP, and attempts in aligned columns using printf.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {printf "%-8s %-16s %3d\n", $1, $2, $5}' auth.log
08:05    172.16.4.21        1
08:12    172.16.4.37        4
08:18    10.10.2.5         12
08:26    192.168.0.44       6
08:31    10.10.2.9          3
08:40    172.16.4.21        2
08:47    192.168.0.51       9
08:55    10.10.2.5          5

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

3. Store the source IP and event in variables, then print them in a sentence.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {src=$2; evt=$3; print "The address " src " generated the event " evt
"."}' auth.log
The address 172.16.4.21 generated the event Login_Success.
The address 172.16.4.37 generated the event Login_Failed.
The address 10.10.2.5 generated the event Port_Scan.
The address 192.168.0.44 generated the event Login_Failed.
The address 10.10.2.9 generated the event Malware_Alert.
The address 172.16.4.21 generated the event File_Access.
The address 192.168.0.51 generated the event Login_Failed.
The address 10.10.2.5 generated the event Port_Scan.

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

4. Calculate the processing time for each record if one attempt takes 2 seconds.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {secs=$5*2; print $2, secs " seconds"}' auth.log
172.16.4.21 2 seconds
172.16.4.37 8 seconds
10.10.2.5 24 seconds
192.168.0.44 12 seconds
10.10.2.9 6 seconds
172.16.4.21 4 seconds
192.168.0.51 18 seconds
10.10.2.5 10 seconds

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

5. Calculate the total number of attempts in the file.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {sum+=$5} END {print "Total attempts =", sum}' auth.log
Total attempts = 42

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

6. Calculate the average number of attempts per record.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {sum+=$5; n++} END {printf "Average attempts per record = %.2f\n",
sum/n}' auth.log
Average attempts per record = 5.25

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

7. Count the number of Login_Failed records.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 && $3=="Login_Failed" {n++} END {print "Login_Failed records =", n}'
auth.log
Login_Failed records = 3

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

8. Start with a review quota of 10 and decrease it by one for every HIGH or CRITICAL record. Print the remaining quota.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'BEGIN {quota=10} NR>1 && ($4=="HIGH" || $4=="CRITICAL") {quota--} END {print
"Remaining quota =", quota}' auth.log
Remaining quota = 6

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

9. Replace Port_Scan with Network_Scan in the event field and print every record in CSV format.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'BEGIN {OFS=","} NR>1 {sub(/^Port_Scan$/, "Network_Scan", $3); print $1, $2,
$3, $4, $5}' auth.log
08:05,172.16.4.21,Login_Success,LOW,1
08:12,172.16.4.37,Login_Failed,MEDIUM,4
08:18,10.10.2.5,Network_Scan,HIGH,12
08:26,192.168.0.44,Login_Failed,HIGH,6
08:31,10.10.2.9,Malware_Alert,CRITICAL,3
08:40,172.16.4.21,File_Access,LOW,2
08:47,192.168.0.51,Login_Failed,MEDIUM,9
08:55,10.10.2.5,Network_Scan,HIGH,5

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

10. Add one to the attempts field of each Login_Failed record and print every record in CSV format.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'BEGIN {OFS=","} NR>1 {if ($3=="Login_Failed") $5=$5+1; print $1, $2, $3, $4,
$5}' auth.log
08:05,172.16.4.21,Login_Success,LOW,1
08:12,172.16.4.37,Login_Failed,MEDIUM,5
08:18,10.10.2.5,Port_Scan,HIGH,12
08:26,192.168.0.44,Login_Failed,HIGH,7
08:31,10.10.2.9,Malware_Alert,CRITICAL,3
08:40,172.16.4.21,File_Access,LOW,2
08:47,192.168.0.51,Login_Failed,MEDIUM,10
08:55,10.10.2.5,Port_Scan,HIGH,5

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

11. Print the source IP and event when severity is HIGH and attempts are greater than 5.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 && $4=="HIGH" && $5>5 {print $2, $3}' auth.log
10.10.2.5 Port_Scan
192.168.0.44 Login_Failed

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

12. Print the source IP and event when severity is HIGH or CRITICAL.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 && ($4=="HIGH" || $4=="CRITICAL") {print $2, $3}' auth.log
10.10.2.5 Port_Scan
192.168.0.44 Login_Failed
10.10.2.9 Malware_Alert
10.10.2.5 Port_Scan

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

13. Use the conditional operator to label each record Repeated if attempts exceed 1; otherwise label it Single.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {label = ($5>1) ? "Repeated" : "Single"; print $2, label}' auth.log
172.16.4.21 Single
172.16.4.37 Repeated
10.10.2.5 Repeated
192.168.0.44 Repeated
10.10.2.9 Repeated
172.16.4.21 Repeated
192.168.0.51 Repeated
10.10.2.5 Repeated

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

14. Use the conditional operator to label each record Investigate if attempts are at least 5; otherwise label it Routine.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {tag = ($5>=5) ? "Investigate" : "Routine"; print $2, tag}' auth.log
172.16.4.21 Routine
172.16.4.37 Routine
10.10.2.5 Investigate
192.168.0.44 Investigate
10.10.2.9 Routine
172.16.4.21 Routine
192.168.0.51 Investigate
10.10.2.5 Investigate

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

15. Use if–else to print Review required for HIGH or CRITICAL records and Routine review for all other records.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {if ($4=="HIGH" || $4=="CRITICAL") msg="Review required"; else
msg="Routine review"; print $2, msg}' auth.log
172.16.4.21 Routine review
172.16.4.37 Routine review
10.10.2.5 Review required
192.168.0.44 Review required
10.10.2.9 Review required
172.16.4.21 Routine review
192.168.0.51 Routine review
10.10.2.5 Review required

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

16. Use if–else if–else to classify attempts as High activity (10 or more), Moderate activity (5–9), or Low activity (fewer than 5).

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {if ($5>=10) cls="High activity"; else if ($5>=5) cls="Moderate
activity"; else cls="Low activity"; print $2, cls}' auth.log
172.16.4.21 Low activity
172.16.4.37 Low activity
10.10.2.5 High activity
192.168.0.44 Moderate activity
10.10.2.9 Low activity
172.16.4.21 Low activity
192.168.0.51 Moderate activity
10.10.2.5 Moderate activity

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

17. Use a for loop to print Check 1 through Check 5.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'BEGIN {for (n=1; n<=5; n++) print "Check", n}'
Check 1
Check 2
Check 3
Check 4
Check 5

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

18. Use a while loop to count down from 10 to 1.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'BEGIN {k=10; while (k>=1) {print k; k--}}'
10
9
8
7
6
5
4
3
2
1

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

19. Use a do–while loop to count down from 10 to 1.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'BEGIN {k=10; do {print k; k--} while (k>=1)}'
10
9
8
7
6
5
4
3
2
1

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

20. Print each record's source IP, event, attempts, and activity classification in aligned columns. At the end, print the total records and total attempts.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {if ($5>=10) cls="High activity"; else if ($5>=5) cls="Moderate
activity"; else cls="Low activity"; printf "%-16s %-15s %-3d %s\n", $2, $3, $5, cls;
recs++; sum+=$5} END {print "Total records:", recs; print "Total attempts:", sum}'
auth.log
172.16.4.21      Login_Success   1   Low activity
172.16.4.37      Login_Failed    4   Low activity
10.10.2.5        Port_Scan       12  High activity
192.168.0.44     Login_Failed    6   Moderate activity
10.10.2.9        Malware_Alert   3   Low activity
172.16.4.21      File_Access     2   Low activity
192.168.0.51     Login_Failed    9   Moderate activity
10.10.2.5        Port_Scan       5   Moderate activity
Total records: 8
Total attempts: 42

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

Login monitoring: A security analyst considers five or more failed attempts suspicious. Write an AWK script that prints the source IP and attempts for matching Login_Failed entries. At the end, print how many suspicious entries were found.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 && $3=="Login_Failed" && $5>=5 {print $2, $5; hits++} END {print
"Suspicious entries found:", hits+0}' auth.log
192.168.0.44 6
192.168.0.51 9
Suspicious entries found: 2

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

Incident summary: Write an AWK script that counts the entries in each severity category and prints a summary in an END block.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {tally[$4]++} END {n=split("LOW MEDIUM HIGH CRITICAL", order, " ");
print "SEVERITY SUMMARY"; for (i=1; i<=n; i++) printf "%-10s %d\n", order[i],
tally[order[i]]+0}' auth.log
SEVERITY SUMMARY
LOW        2
MEDIUM     2
HIGH       3
CRITICAL   1

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

Processing estimate: Assume each attempt requires 2 seconds of analyst review. Calculate the review time for each entry and the total review time for the file. Use printf to present the results neatly.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {t=$5*2; printf "%-8s %-16s %3d seconds\n", $1, $2, t; all+=t} END
{printf "Total review time: %d seconds\n", all}' auth.log
08:05    172.16.4.21        2 seconds
08:12    172.16.4.37        8 seconds
08:18    10.10.2.5         24 seconds
08:26    192.168.0.44      12 seconds
08:31    10.10.2.9          6 seconds
08:40    172.16.4.21        4 seconds
08:47    192.168.0.51      18 seconds
08:55    10.10.2.5         10 seconds
Total review time: 84 seconds

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

Log normalization: A monitoring tool requires the event name Failed_Login instead of Login_Failed. Update the matching field and print all records as CSV. The source file should remain unchanged.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'BEGIN {OFS=","} NR>1 {if ($3=="Login_Failed") $3="Failed_Login"; print $1,
$2, $3, $4, $5}' auth.log
08:05,172.16.4.21,Login_Success,LOW,1
08:12,172.16.4.37,Failed_Login,MEDIUM,4
08:18,10.10.2.5,Port_Scan,HIGH,12
08:26,192.168.0.44,Failed_Login,HIGH,6
08:31,10.10.2.9,Malware_Alert,CRITICAL,3
08:40,172.16.4.21,File_Access,LOW,2
08:47,192.168.0.51,Failed_Login,MEDIUM,9
08:55,10.10.2.5,Port_Scan,HIGH,5

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ grep -c "Login_Failed" auth.log
3

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```

Priority queue: Assign each entry a priority: Immediate for CRITICAL, Soon for HIGH, and Routine for all others. Print the time, source IP, event, and assigned priority.

```
anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ awk 'NR>1 {if ($4=="CRITICAL") pr="Immediate"; else if ($4=="HIGH") pr="Soon";
else pr="Routine"; print $1, $2, $3, pr}' auth.log
08:05 172.16.4.21 Login_Success Routine
08:12 172.16.4.37 Login_Failed Routine
08:18 10.10.2.5 Port_Scan Soon
08:26 192.168.0.44 Login_Failed Soon
08:31 10.10.2.9 Malware_Alert Immediate
08:40 172.16.4.21 File_Access Routine
08:47 192.168.0.51 Login_Failed Routine
08:55 10.10.2.5 Port_Scan Soon

anonymous@anonymous-supreme in ~/awk_lab via v3.14.7
❯ _
```